Relatório de execução

Gabriel Oliveira de Jesus e Ives Yudy Omura Maruiti — 22/10/2026

Implementamos em conjunto uma aplicação de terminal em Python com SQLite, organizada em modelos, serviços de regras de negócio, repositório e interface. A solução contempla agenda, reservas individuais ou coletivas, limite semanal, cancelamentos, exceções analisadas pelo ADMIN, avisos internos, histórico, presença, faltas, avaliações, dúvidas agrupadas e relatório mensal de horas restrito ao ADMIN.

Ajustamos o cronograma durante a execução. O plano da Parte 1 tem data de 16/10; versionamos o escopo inicial em 18/10, antes do primeiro código de perfis e agenda. Registramos a integração de reservas persistentes e cancelamentos em 21/10, assim como as exceções e os recursos previstos para 20/10. Portanto, consolidamos parte da implementação depois das datas planejadas. Usamos as datas dos commits como referência de registro, sem confundi-las com o início de cada atividade. Preparamos a documentação final e realizamos a conferência em 22/10. Acrescentamos as cinco histórias detalhadas nesta etapa.

Refinamos os nomes do projeto inicial para inglês e a dúvida passou a pertencer à reserva. O limite semanal tornou-se um critério separado (CA5). As regras recebem parâmetros; entrada e saída ficam no terminal. Reservas e cancelamentos usam transações para manter capacidade, histórico e avisos consistentes. A presença passou a fundamentar o cálculo de horas e a permissão de avaliar.

Executamos os 90 testes unittest com sucesso na conferência local, abrangendo os critérios de aceitação, permissões, persistência, concorrência, cancelamento, exceções, presença, feedback, relatórios e fluxos do terminal. A entrega inclui o código, testes, documentos editáveis, PDF consolidado e o plano da Parte 1. A seleção de perfis é demonstrativa, sem autenticação real; avisos são internos. As datas usam horário local sem fuso. Preservamos o projeto inicial como registro histórico, com referências à autoria conjunta, e distinguimos suas previsões do estado final.

Histórias de usuário

H1 — Reservar atendimento — Alta (escolhida)

Como aluno, quero escolher uma disciplina, consultar horários livres e reservar uma monitoria de 30 minutos, para organizar meu atendimento sem combinar por mensagem.

Justificativa: é o fluxo central, depende de capacidade consistente e fundamenta os demais.

Trecho: “A ideia é que o aluno escolha a disciplina, veja os horários livres e reserve um atendimento.”

1. CA1: listar apenas vagas futuras da disciplina escolhida com capacidade restante.
2. CA2: gravar a reserva e consumir uma posição da capacidade da vaga.
3. CA3: recusar vaga lotada ou inexistente sem gravar reserva.
4. CA4: exigir o conteúdo da dúvida e permitir sua consulta pelos alunos da mesma vaga.
5. CA5: limitar a duas reservas ativas por aluno/disciplina/semana, com limite configurável; semanas de segunda a domingo, cancelamento libera a posição e o limite.

Rastreabilidade: tests/test_criteria.py cobre sucesso e falha de CA1–CA5; tests/test_bookings.py cobre persistência e concorrência.

H2 — Organizar agenda e atendimentos em grupo — Alta

Como monitor, quero publicar vagas por disciplina e definir capacidade individual ou coletiva, para organizar os atendimentos e reunir alunos com dúvidas relacionadas.

Justificativa: disponibilizar vagas é pré-requisito para reservas.

Trecho: “Cada atendimento dura 30 minutos, mas alguns monitores preferem atender em grupo”.

1. Somente MONITOR publica vagas próprias, com disciplina não vazia e início futuro.
2. Toda vaga dura 30 minutos e possui capacidade inteira positiva.
3. Recusar sobreposição de vagas do mesmo monitor, inclusive em disciplinas diferentes.
4. Permitir horários adjacentes e horários simultâneos de monitores diferentes.
5. O monitor responsável pode consultar as dúvidas dos alunos de suas vagas.

Rastreabilidade: tests/test_schedule.py, tests/test_permissions.py e tests/test_bookings.py.

H3 — Acompanhar resultados e horas — Média

Como ADMIN, quero consultar histórico, dúvidas agrupadas, avaliações e horas mensais por monitor, para orientar melhorias e apoiar o acompanhamento das bolsas.

Justificativa: usa os registros de atendimentos e exige restrição das informações de bolsa.

Trecho: “A coordenação precisa de um relatório mensal com as horas atendidas por cada monitor”.

1. Restringir relatórios mensais de horas e agrupamento de dúvidas ao ADMIN.
2. Contar 0,5 hora por vaga encerrada com pelo menos um aluno presente, sem multiplicar pelo grupo.
3. Agrupar dúvidas das reservas ativas por disciplina e conteúdo, normalizando caixa e espaços.
4. Aluno com presença pode avaliar uma vez após o fim, com nota inteira de 1 a 5.
5. Aluno vê seu histórico e avaliações; monitor vê os de suas vagas; ADMIN vê todos.

Rastreabilidade: tests/test_reports.py, tests/test_feedback.py, tests/test_attendance.py e tests/test_permissions.py.

H4 — Registrar presença e avisar faltas — Média

Como monitor, quero registrar presença ou falta e gerar avisos, para reduzir horários perdidos sem bloquear automaticamente os alunos.

Justificativa: responsabiliza pelas ausências sem adotar bloqueio não consensual.

Trecho: “Quando o aluno reserva e não aparece, o horário fica perdido.”

1. Apenas monitor da vaga ou ADMIN registra presença após o início do atendimento.
2. Cada reserva ativa recebe uma única marcação; canceladas não recebem presença.
3. Cada falta gera aviso interno ao aluno.
4. A terceira falta no semestre gera aviso adicional ao aluno e ao ADMIN.
5. Somar faltas entre disciplinas e reiniciar a contagem a cada semestre civil.

Rastreabilidade: tests/test_attendance.py e tests/test_cli.py.

H5 — Cancelar com transparência — Alta

Como aluno ou monitor responsável, quero cancelar uma reserva com aviso aos envolvidos e pedir exceção quando necessário, para evitar deslocamentos sem atendimento.

Justificativa: cancelamentos inesperados geram conflito e exigem uma regra clara.

Trecho: “Os alunos reclamam que só descobrem o cancelamento quando chegam na sala.”

1. Aluno cancela sua reserva; monitor cancela reservas de suas vagas, com antecedência mínima de 72 horas.
2. Dentro das 72 horas, aluno ou monitor pode pedir exceção com motivo ao ADMIN.
3. Apenas ADMIN aprova ou recusa uma única vez; recusa exige justificativa.
4. Exceção aprovada permite cancelar antes do início; ADMIN pode cancelar diretamente com justificativa fora do prazo.
5. Preservar o registro cancelado, liberar capacidade e registrar avisos internos aos envolvidos.

Rastreabilidade: tests/test_exceptions.py, tests/test_bookings.py e tests/test_cli.py.

História escolhida (Alta): Como aluno, eu quero escolher uma disciplina, ver os horários livres e reservar um atendimento de 30 minutos, para não combinar por mensagem nem chegar e encontrar o monitor ocupado ou ausente.

Critérios de aceitação

1. Ao escolher uma disciplina, o sistema lista apenas os horários com vaga disponível do monitor daquela disciplina. “A ideia é que o aluno escolha a disciplina, veja os horários livres e reserve um atendimento “
2. Ao reservar um horário livre, a reserva é registrada e consome uma posição da vaga; a capacidade da vaga é definida pelo monitor (1 = individual, maior que 1 = grupo). “Cada atendimento dura 30 minutos, mas alguns monitores preferem atender em grupo, quando várias pessoas têm a mesma dúvida. “ “Na semana passada, um monitor de Cálculo tinha três alunos marcados no mesmo horário, enquanto outros passaram a tarde inteira sem ninguém. “
3. O sistema recusa a reserva de um horário sem vaga restante (lotado), com erro claro. “Isso tem gerado confusão. Na semana passada, um monitor de Cálculo tinha três alunos marcados no mesmo horário, enquanto outros passaram a tarde inteira sem ninguém. “
4. Ao reservar, o aluno informa o conteúdo (ementa) da dúvida; a reserva sem conteúdo é recusada, e o conteúdo fica visível aos demais alunos da mesma vaga.

“Os professores das disciplinas gostariam de saber quais conteúdos geram mais dúvidas, para reforçar em aula. “

“...mas alguns monitores preferem atender em grupo, quando várias pessoas têm a mesma dúvida. “

Funções e classes e suas responsabilidades

- Vaga: representa um horário de 30 min de um monitor, com disciplina, início, ementa e capacidade (1 para individual, mais de 1 para grupo).
- Reserva: liga um aluno a uma vaga e guarda a data da reserva.
- RepositorioAgendamento: única parte que fala com o SQLite. Cria as tabelas, grava e consulta vagas e reservas. Os testes usam banco de memória.
- ServicoAgendamento: concentra as regras de negócio.

listar_horarios_livres(disciplina): devolve só vagas com capacidade ainda não esgotada (CA1).

valida as regras e registra a reserva (reservar(aluno, vaga)CA2).

Auxiliar que conta as reservas do aluno na disciplina dentro da semana da vaga, usada para aplicar o limite (CA4).

- Erros próprios: vaga inexistente, vaga lotada e limite semanal excedido, falta de resposta do monitor. Os testes de falha verificam essas exceções.

Nenhuma função lê dados digitados: tudo recebe parâmetros, então os testes chamam direto (requisito mínimo 3).

Nomenclatura dos testes que verifica cada critério :

1. Teste_Lista_Disponivel
2. Teste_Consome_Vaga
3. Teste_Recusa_Reserva
4. Teste_Ementa_Feita

Nomenclatura das Falha/exceção dos critérios:

1. Teste_Lista_Vazia

2. Teste_Consumo_Inexistente
3. Teste_Erro_Reserva
4. Teste_Sem_Ementa

Refinamento antes da integração — 18/10/2026

Os nomes da implementação serão em inglês. As definições abaixo corrigem as ambiguidades das responsabilidades descritas acima:

- User e Role: usuário e perfil (ADMIN, MONITOR, STUDENT).
- AppointmentSlot: vaga de 30 minutos, monitor, disciplina, início e capacidade. O conteúdo da dúvida não pertence à vaga.
- Booking: reserva com aluno, vaga, conteúdo obrigatório da dúvida (question) e data de criação. O conteúdo será consultável pelos alunos da mesma vaga.
- ScheduleRepository: contrato da agenda; InMemoryScheduleRepository é o adaptador provisório. Implementaremos o adaptador SQLite.
- ScheduleService.create_slot(actor, subject, starts_at, capacity=1) publica uma vaga própria do monitor, validando capacidade e conflitos de horário.
- ScheduleService.list_slots(subject=None) consulta a agenda futura publicada. A filtragem por capacidade restante (CA1) depende da integração com reservas.
- Futuro BookingService.book(student, slot_id, question): valida e registra a reserva. Recusa vaga inexistente, lotação, conteúdo vazio e limite semanal.

CA5 adicional: o limite semanal é configurável, com padrão de duas reservas por aluno/disciplina na semana da vaga (segunda a domingo). Recusar uma terceira reserva nessa semana; aceitar em outra semana ou disciplina, havendo capacidade. Este critério é independente do CA4, que trata do conteúdo da dúvida.

Testes futuros de reservas: test_booking_consumes_capacity, test_full_slot_rejected, test_unknown_slot_rejected, test_question_required, test_question_visible_to_slot_students, test_weekly_limit_rejected, test_other_week_allowed e test_other_subject_allowed. Implementaremos esses testes na etapa de reservas. A falta de resposta do monitor não integra os critérios de reservas desta etapa.

Os testes já implementados em tests/ verificam inicialmente agenda, perfis, permissões e fluxo do terminal. Ainda não comprovam CA1–CA5 integrados.

Documentação final

Execução e ambiente

Python 3.10 ou superior; SQLite e unittest da biblioteca padrão. Não é necessário instalar dependências. Na pasta do projeto, execute python3 -m monitoring. Para testes: python3 -m unittest discover -v. O banco monitoring.sqlite3 é criado no diretório atual. Excluir esse arquivo remove os dados da demonstração.

Roteiro de uso

Selecione 2 (MONITOR), opção 1, informe disciplina, data futura no formato AAAA-MM-DD HH:MM e capacidade. Volte com 0 e selecione 3 (STUDENT). Consulte a agenda (2), reserve pelo id (3) e informe a dúvida. Consulte suas reservas (4), dúvidas da vaga (7) e avisos (6). Cancele pela opção 5. Com menos de 72 horas, peça exceção (8); ADMIN analisa pela opção 9. A aprovação permite ao interessado efetuar o cancelamento pela opção 5.

Após o início, o monitor registra presença/falta (14). O histórico fica em 15. Depois do término, aluno presente avalia em 10; avaliações são consultadas em 11. ADMIN consulta horas mensais em 12 e dúvidas agrupadas em 13. Para demonstrar presença sem esperar, use os cenários com relógio controlado dos testes; o menu não permite criar vaga passada nem manipular o relógio.

Arquitetura e dados

models.py define usuários, perfis, vagas, reservas, avisos, pedidos de exceção, avaliações e presença. schedule.py trata agenda e conflitos; bookings.py trata capacidade, limites e cancelamento. Serviços separados cuidam de exceções, presença, feedback e relatórios. sqlite_repository.py é a única camada SQL. cli.py cuida dos menus. Não há serviços externos ou credenciais necessárias.

Tabelas: slots, bookings, notifications, exception_requests, feedback e attendance. Reservas referenciam vagas; presença e avaliação são únicas por reserva. Índices restringem reserva ativa duplicada e pedidos de exceção abertos. Transações BEGIN IMMEDIATE protegem alterações de reserva e cancelamento. Cancelados permanecem no histórico; vagas de grupo contam uma vez nas horas.

Verificação e limites

A suíte tem 90 testes; veja docs/testes.md para a cobertura por critério. A seleção de usuário simula os três perfis e não é login seguro. Os usuários demonstrativos têm ids fixos; não há cadastro ou dados de contato. O histórico registra atendimentos e presença, sem medir desempenho acadêmico. Feedback é do aluno sobre o atendimento. Avisos não são enviados por e-mail. A interface é de terminal, sem versão móvel. Conferimos a entrega com base nos documentos de requisitos e no plano de execução.

Mantivemos o escopo inicial como registro da concepção anterior à implementação, ajustando as referências de autoria para refletir o trabalho conjunto. Expressões como “futuro” nesse arquivo descrevem aquele momento; a situação atual está no README e neste documento. O relatório de execução explica os refinamentos sem reescrever o registro original.

Cobertura dos testes

Execução: python -m unittest discover -v (todos usam SQLite em memória ou arquivo temporário).

Critérios de aceitação

Critério	Sucesso	Falha/exceção	Arquivo
CA1 Lista só vagas livres da disciplina	Teste_Lista_Disponivel	Teste_Lista_Vazia	tests/test_criteria.py
CA2 Reserva registrada e consome vaga	Teste_Consome_Vaga	Teste_Consumo_Inexistente	tests/test_criteria.py
CA3 Recusa de vaga lotada	Teste_Recusa_Reserva	Teste_Erro_Reserva	tests/test_criteria.py
CA4 Ementa obrigatória e visível na vaga	Teste_Ementa_Feita	Teste_Sem_Ementa	tests/test_criteria.py
CA5 Limite semanal configurável	Teste_Limite_Semanal_Aceita	Teste_Limite_Semanal	tests/test_criteria.py

Na saída com -v, cada teste aparece com o nome do escopo.

Demais regras

Regra	Arquivo
Agenda, conflitos e perfis	tests/test_schedule.py
Reservas, cancelamento (72 h), concorrência e persistência	tests/test_bookings.py
Exceções de cancelamento e avisos	tests/test_exceptions.py
Presença, faltas por semestre, avisos e histórico	tests/test_attendance.py
Feedback dos alunos	tests/test_feedback.py
Relatório mensal de horas e dúvidas agrupadas	tests/test_reports.py
Matriz de permissões: cada operação × cada perfil	tests/test_permissions.py
Fluxos do terminal	tests/test_cli.py

Permissões verificadas

tests/test_permissions.py executa cada operação com os cinco usuários (ADMIN, dois monitores, dois alunos): quem deve ser barrado precisa receber PermissionDeniedError e a tentativa não pode deixar vaga, reserva, solicitação ou avaliação gravada; quem pode tentar não pode ser barrado por permissão. Também confere o que cada perfil enxerga em reservas e avaliações.